

The Practical Python Handbook

A friendly, from-scratch guide to Python — for beginners building real understanding, one concept at a time.

This handbook is an original, freely shareable educational resource. It covers Python fundamentals through to intermediate topics, with runnable code examples throughout. It's designed to work well both as a standalone reference and as source material for study tools (such as retrieval-based tutoring systems).

Table of Contents

- 1. Getting Started with Python
- 2. Variables and Data Types
- 3. Operators
- 4. Strings in Depth
- 5. Control Flow
- 6. Data Structures: Lists, Tuples, Sets, Dictionaries
- 7. Functions
- 8. Comprehensions
- 9. Error Handling
- 10. File Handling
- 11. Object-Oriented Programming
- 12. Modules and Packages
- 13. Iterators and Generators
- 14. Decorators
- 15. The Standard Library: Useful Modules
- 16. Best Practices and Common Pitfalls

Chapter 1: Getting Started with Python

1.1 What Is Python?

Python is a high-level, general-purpose programming language known for its readable syntax and broad ecosystem of libraries. It is used across web development, data science, automation, scripting, and education. Python is an interpreted language, meaning code is executed line by line by the Python interpreter rather than compiled into machine code ahead of time.

1.2 Installing Python

Most operating systems can run Python after installing it from the official source at python.org, or through a package manager. You can confirm an installation by opening a terminal and running:

```
python --version
# or, on some systems:
python3 --version
```

1.3 Your First Program

By long-standing tradition, the first program written in any language prints a short greeting. In Python, this takes a single line:

```
print("Hello, World!")
```

Save this line in a file named **hello.py** and run it from a terminal with **python hello.py**. The word **print** is a built-in function — a reusable block of behavior — that outputs text to the screen.

1.4 Comments

Comments are notes in your code that Python ignores when running the program. They exist purely to help humans understand the code.

```
# This is a single-line comment
print("This line runs") # comments can follow code too

"""
This is a multi-line string,
often used as a block comment
or documentation.
"""
```

Chapter 2: Variables and Data Types

2.1 Variables

A variable is a name that refers to a value stored in memory. Python does not require you to declare a variable's type in advance — the type is inferred from the value assigned to it.

```
age = 25
name = "Alice"
height = 1.68
is_student = True
```

Variable names in Python must start with a letter or underscore, can contain letters, numbers, and underscores, and are case-sensitive (**age** and **Age** are different variables).

2.2 Core Data Types

- **int** — whole numbers, e.g. 42, -7
- **float** — decimal numbers, e.g. 3.14, -0.5
- **str** — text, e.g. "hello"
- **bool** — True or False
- **NoneType** — represents the absence of a value, written None

2.3 Checking and Converting Types

```
x = 10
print(type(x))          # <class 'int'>

y = str(x)
print(type(y))          # <class 'str'>

z = int("42")
print(z + 8)             # 50
```

Note: Converting a non-numeric string to a number, like `int("abc")`, raises a `ValueError`. Always validate input before converting it.

2.4 Dynamic Typing

A variable's type can change over its lifetime, because the name is simply reassigned to a new value:

```
value = 5                # value is an int
value = "five"           # now value is a str
value = [5]               # now value is a list
```

Chapter 3: Operators

3.1 Arithmetic Operators

```
a, b = 7, 2

print(a + b)    # 9    addition
print(a - b)    # 5    subtraction
print(a * b)    # 14   multiplication
print(a / b)    # 3.5  division (always returns a float)
print(a // b)   # 3    floor division (rounds down)
print(a % b)    # 1    modulo (remainder)
print(a ** b)   # 49   exponentiation
```

3.2 Comparison Operators

These compare two values and return a boolean result.

```
print(5 == 5)   # True
print(5 != 3)   # True
print(5 > 3)     # True
print(5 < 3)     # False
print(5 >= 5)   # True
print(5 <= 4)   # False
```

3.3 Logical Operators

```
print(True and False) # False
print(True or False)  # True
print(not True)        # False
```

3.4 Assignment Operators

```
count = 10
count += 5    # same as: count = count + 5    -> 15
count -= 3    # -> 12
count *= 2    # -> 24
count /= 5    # -> 4
```

3.5 Identity and Membership Operators

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]

print(b is a)      # True (same object in memory)
print(c is a)      # False (equal values, different objects)
print(c == a)      # True (values are equal)

print(2 in a)      # True
print(9 in a)      # False
```

Chapter 4: Strings in Depth

4.1 Creating Strings

```
single = 'hello'
double = "hello"
multiline = """This string
spans multiple
lines"""
```

4.2 String Indexing and Slicing

Strings are sequences of characters, and each character has a position (index) starting from 0.

```
word = "Python"
print(word[0])      # P
print(word[-1])     # n (last character)
print(word[0:3])    # Pyt (characters at index 0, 1, 2)
print(word[2:])     # thon
print(word[::-1])   # nohtyP (reversed)
```

4.3 Common String Methods

```
" hi ".strip()      # "hi"          removes surrounding whitespace
"hi".upper()        # "HI"
"HI".lower()        # "hi"
"a,b,c".split(",")  # ['a', 'b', 'c']
"-".join(["a", "b", "c"]) # "a-b-c"
"hello".replace("l", "L") # "heLlo"
"hello".startswith("he") # True
"hello".find("l")      # 2 (index of first match, -1 if not found)
```

4.4 f-strings (Formatted String Literals)

f-strings, introduced in Python 3.6, let you embed expressions directly inside a string using curly braces.

```
name = "Maya"
age = 30
print(f"{name} is {age} years old")
# Output: Maya is 30 years old

print(f"Next year she will be {age + 1}")
# Output: Next year she will be 31
```

4.5 String Immutability

Note: Strings in Python are **immutable** — once created, a string's content cannot be changed in place. Methods like `.upper()` return a **new** string rather than modifying the original.

Chapter 5: Control Flow

5.1 if / elif / else

```
age = 20

if age < 13:
    print("Child")
elif age < 20:
    print("Teenager")
else:
    print("Adult")
```

Python uses **indentation** (typically 4 spaces) to define blocks of code, rather than curly braces or keywords. Consistent indentation is required — mixing tabs and spaces, or indenting incorrectly, causes an `IndentationError`.

5.2 for Loops

```
for fruit in ["apple", "banana", "cherry"]:
    print(fruit)

for i in range(5):          # 0, 1, 2, 3, 4
    print(i)

for i in range(2, 10, 2):   # 2, 4, 6, 8
    print(i)
```

5.3 while Loops

```
count = 0
while count < 3:
    print(count)
    count += 1
# prints 0, 1, 2
```

5.4 break, continue, and else on Loops

```
for i in range(10):
    if i == 3:
        continue    # skip this iteration
    if i == 6:
        break        # exit the loop entirely
    print(i)
# prints 0, 1, 2, 4, 5
```

Note: Loops in Python can have an `else` clause, which runs only if the loop completes without hitting a `break`. This is uncommon but occasionally useful for search-style loops.

Chapter 6: Data Structures

6.1 Lists

A list is an ordered, mutable (changeable) collection of items.

```
fruits = ["apple", "banana", "cherry"]
fruits.append("date")      # add to the end
fruits.insert(1, "kiwi")   # insert at position 1
fruits.remove("banana")    # remove by value
fruits.pop()               # remove and return the last item
fruits.sort()              # sort in place
print(len(fruits))         # number of items
```

6.2 Tuples

A tuple is like a list but **immutable** — once created, it cannot be changed. Tuples are often used for fixed collections of values, like coordinates.

```
point = (3, 4)
x, y = point    # unpacking
print(x, y)     # 3 4
```

6.3 Sets

A set is an unordered collection of **unique** items — duplicates are automatically removed.

```
numbers = {1, 2, 2, 3, 3, 3}
print(numbers)      # {1, 2, 3}

a = {1, 2, 3}
b = {2, 3, 4}
print(a | b)        # union      {1, 2, 3, 4}
print(a & b)        # intersection {2, 3}
print(a - b)        # difference  {1}
```

6.4 Dictionaries

A dictionary stores data as key-value pairs, allowing fast lookup by key.

```
student = {"name": "Alice", "age": 22, "major": "Physics"}

print(student["name"])      # Alice
student["age"] = 23         # update a value
student["gpa"] = 3.8        # add a new key

for key, value in student.items():
    print(key, "->", value)
```

Note: Use `dict.get(key, default)` instead of `dict[key]` when a key might not exist — it returns a fallback value instead of raising a `KeyError`.

Chapter 7: Functions

7.1 Defining and Calling Functions

```
def greet(name):  
    return f"Hello, {name}!"  
  
message = greet("Sam")  
print(message)    # Hello, Sam!
```

7.2 Default and Keyword Arguments

```
def power(base, exponent=2):  
    return base ** exponent  
  
print(power(5))           # 25    (uses default exponent)  
print(power(5, 3))        # 125  
print(power(exponent=3, base=5)) # 125 (order doesn't matter with keywords)
```

7.3 *args and **kwargs

These let a function accept a variable number of positional or keyword arguments.

```
def total(*numbers):  
    return sum(numbers)  
  
print(total(1, 2, 3, 4))    # 10  
  
def describe(**details):  
    for key, value in details.items():  
        print(f"{key}: {value}")  
  
describe(name="Rex", species="dog")
```

7.4 Scope

Variables defined inside a function are **local** to it and don't exist outside. Variables defined at the top level of a script are **global**.

```
x = 10    # global  
  
def show():  
    x = 5    # local, separate from the global x  
    print(x)  
  
show()      # 5  
print(x)    # 10 (unchanged)
```

Chapter 8: Comprehensions

8.1 List Comprehensions

A concise way to build a list by transforming or filtering another sequence.

```
squares = [x ** 2 for x in range(6)]
print(squares)    # [0, 1, 4, 9, 16, 25]

evens = [x for x in range(10) if x % 2 == 0]
print(evens)      # [0, 2, 4, 6, 8]
```

8.2 Dictionary and Set Comprehensions

```
squares_dict = {x: x ** 2 for x in range(5)}
print(squares_dict)    # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

unique_lengths = {len(word) for word in ["hi", "bye", "ok", "yo"]}
print(unique_lengths)  # {2, 3}
```

Note: Comprehensions are often faster and more readable than an equivalent for loop with `.append()`, but very nested comprehensions can hurt readability — a regular loop is sometimes the clearer choice.

Chapter 9: Error Handling

9.1 try / except

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
```

9.2 Handling Multiple Exception Types

```
try:
    value = int(input("Enter a number: "))
    result = 10 / value
except ValueError:
    print("That was not a valid number")
except ZeroDivisionError:
    print("Cannot divide by zero")
else:
    print(f"Result: {result}")    # runs only if no exception occurred
finally:
    print("Done attempting the calculation")    # always runs
```

9.3 Raising Exceptions

```
def set_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    return age
```

Note: Catch the most specific exception type you can. Broad, catch-all `except :` blocks can hide real bugs by silently swallowing errors you didn't anticipate.

Chapter 10: File Handling

10.1 Reading and Writing Files

```
# Writing to a file
with open("notes.txt", "w") as f:
    f.write("Hello, file!\n")

# Reading a file
with open("notes.txt", "r") as f:
    content = f.read()
    print(content)
```

The with statement ensures the file is automatically closed when the block finishes, even if an error occurs.

10.2 Reading Line by Line

```
with open("notes.txt", "r") as f:
    for line in f:
        print(line.strip())
```

10.3 File Modes

- 'r' — read (default; error if file doesn't exist)
- 'w' — write (creates file, overwrites existing content)
- 'a' — append (adds to the end of existing content)
- 'x' — exclusive creation (fails if file already exists)

Chapter 11: Object-Oriented Programming

11.1 Classes and Objects

A class is a blueprint for creating objects. Each object created from a class is called an **instance**.

```
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed

    def bark(self):
        return f"{self.name} says Woof!"

my_dog = Dog("Rex", "Labrador")
print(my_dog.bark())  # Rex says Woof!
```

`__init__` is a special method called a **constructor** — it runs automatically when a new instance is created, and is used to set up the object's initial state. `self` refers to the instance itself.

11.2 Inheritance

A class can inherit attributes and methods from another class, letting you reuse and extend behavior.

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return f"{self.name} makes a sound"

class Cat(Animal):
    def speak(self):
        return f"{self.name} says Meow"

a = Animal("Generic")
c = Cat("Whiskers")
print(a.speak())  # Generic makes a sound
print(c.speak())  # Whiskers says Meow
```

Here, `Cat` overrides the `speak` method from `Animal` — this is called **method overriding**.

11.3 Encapsulation Conventions

Python doesn't enforce strict private attributes, but uses naming conventions to signal intent:

- `name` — public, freely accessible
- `_name` — 'protected' by convention, meant for internal use
- `__name` — 'name-mangled', harder to access accidentally from outside the class

Chapter 12: Modules and Packages

12.1 Importing Modules

```
import math
print(math.sqrt(16))    # 4.0

from math import sqrt, pi
print(sqrt(16))         # 4.0
print(pi)               # 3.14159...

import math as m
print(m.sqrt(16))       # 4.0
```

12.2 Writing Your Own Module

Any .py file can be imported as a module. If you have a file `helpers.py` containing:

```
def double(x):
    return x * 2
```

...you can use it from another file in the same folder with:

```
import helpers
print(helpers.double(5))    # 10
```

12.3 Packages

A package is a folder containing multiple related modules, along with a special `__init__.py` file that marks the folder as importable. This is how larger projects organize related code into a hierarchy.

Chapter 13: Iterators and Generators

13.1 Iterables and Iterators

An **iterable** is anything you can loop over (lists, strings, dictionaries). Internally, Python gets an **iterator** from it — an object that produces one value at a time via a `__next__` method.

13.2 Generators

A generator is a special function that produces values one at a time, pausing its state between each value, using `yield` instead of `return`.

```
def countdown(n):
    while n > 0:
        yield n
        n -= 1

for number in countdown(3):
    print(number)
# prints 3, 2, 1
```

Note: Generators are memory-efficient because they produce values on demand instead of building an entire list in memory upfront — useful for working with very large or infinite sequences.

13.3 Generator Expressions

```
squares = (x ** 2 for x in range(1000000))    # created instantly, no memory used yet
print(next(squares))    # 0
print(next(squares))    # 1
```

Chapter 14: Decorators

14.1 Functions as First-Class Objects

In Python, functions can be passed around like any other value — assigned to variables, passed as arguments, and returned from other functions.

14.2 Writing a Decorator

A decorator is a function that wraps another function to add behavior before or after it runs, without modifying the original function's code.

```
def log_call(func):
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        result = func(*args, **kwargs)
        print(f"{func.__name__} finished")
        return result
    return wrapper

@log_call
def add(a, b):
    return a + b

print(add(2, 3))
# Calling add
# add finished
# 5
```

The `@log_call` syntax is shorthand for `add = log_call(add)`.

Chapter 15: The Standard Library

Python ships with a large collection of built-in modules covering common tasks, so you often don't need an external library for basic needs. A few of the most frequently used:

os — Operating System Interfaces

```
import os
print(os.getcwd())      # current working directory
print(os.listdir("."))  # list files in current directory
```

datetime — Dates and Times

```
from datetime import datetime
now = datetime.now()
print(now.year, now.month, now.day)
```

json — Working with JSON Data

```
import json
data = {"name": "Alice", "age": 30}
text = json.dumps(data)      # convert dict to JSON string
parsed = json.loads(text)    # convert JSON string back to dict
```

random — Random Number Generation

```
import random
print(random.randint(1, 10))  # random integer from 1 to 10
print(random.choice(["a", "b", "c"])) # random item from a list
```

collections — Specialized Container Types

```
from collections import Counter
counts = Counter(["a", "b", "a", "c", "a"])
print(counts)  # Counter({'a': 3, 'b': 1, 'c': 1})
```

Chapter 16: Best Practices and Common Pitfalls

16.1 Follow PEP 8

PEP 8 is Python's official style guide. Key conventions: use 4 spaces per indentation level, name variables and functions in snake_case, name classes in PascalCase, and keep lines under about 79 characters where practical.

16.2 Avoid Mutable Default Arguments

A common pitfall: using a mutable object like a list as a default argument value. The default is created only once, and shared across every call.

```
# Problematic:
def add_item(item, items=[]):
    items.append(item)
    return items

print(add_item("a"))    # ['a']
print(add_item("b"))    # ['a', 'b'] <- unexpected! The list persisted.

# Fixed:
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items
```

16.3 Use Meaningful Names

Prefer total_price over tp, and is_valid over flag. Code is read far more often than it's written.

16.4 Don't Compare to True/False Directly

```
# Prefer:
if is_active:
    ...

# Over:
if is_active == True:
    ...
```

16.5 Use Virtual Environments

Each project should generally have its own isolated environment for dependencies, to avoid version conflicts between projects.

```
python -m venv venv
source venv/bin/activate    # on macOS/Linux
```

```
venv\Scripts\activate      # on Windows  
pip install some-package
```

16.6 Closing Thought

Python rewards clarity. When in doubt, favor the version of your code that reads most simply and directly — readability is treated as a core value in the language's own design philosophy, sometimes summed up as: 'There should be one — and preferably only one — obvious way to do it.'